一、原始数据与字段含义

本次实验先使用小米运动健康中导出数据进行分析，选取了同一分钟的步数和心率进行聚类处理，并希望通过这样的方式了解数据集中是否存在一些可以被归类的潜在模式。经过聚类处理后发现，样本可以被分为步数较少且心率偏低和步数较多且心率较高的两组。

现在先说明原始数据的处理方法。小米导出的文件中包含了健康明细和每日汇总等文件，本文的数据分析使用的是来自于健康明细那个文件的数据集，并放置在 `data/fitness_data.csv` 中。这个数据集的形状是 71728 行和 6 列，里面包含了 Uid、Sid、Key、Time、Value 和 UpdateTime 等字段。其中，Uid 是我的账号 Uid，Sid 是我的小米手环的设备 ID，Key 是记录的类型，Time 和 UpdateTime 分别保存了记录的生成与更新的时间。

这意味着真正保存了健康数据的字段是 Value，它是一个 JSON 字符串，并且由下面几种键构成。每个键对应的一种示例值，以及每个键的意思如下表 1 所示。

表 1　Key 与 Value 的对应含义

Key	Value 示例	示例含义
heart_rate	"bom":92	心率为 92 次/分钟
steps	"steps":7,"distance":5	步数为 7 步且距离为 5 米
spo2	"spo2":95	血氧饱和度为 95%
stress	"stress":34	设备给出的压力评分为 34
sleep	"duration":41	改段睡眠记录时长为 41 分钟

本次使用小米手环 8 记录的步数和心率数据，分别有 2056 条和 22612 条数据。原始文件中还包含了来自手机记录的步数数据，这部分数据可能与手环的记录重合，所以在实际处理中并没有使用这部分的数据。接下来，只需要将同一分钟的步数和心率两两配对并进行聚类分析，就可以比较走动较多和较少时的心率的变化情况。

二、二维数据的提取与处理

本次选择 steps 和 bpm 两个指标进行聚类处理，其中，前者反映了对应分钟时刻手环记录到的步数，后者则是同时刻的每分钟心跳次数。选择这两个指标，是为了在观察活动多少的同时，了解相应的心率水平。例如，步数差不多的记录可能会有较大差别的心率，而步数较少时也可能出现更高的心率。所以，通过两两组合这两种数据，就能比较不同走动情况下心率的差异，更具体地比较不同记录的特点，为后续分析我的活动情况提供参考。

整理数据时，先通过 Key 找出步数和心率记录，再根据 Sid 筛选出来自手环的所有记录。然后用 `json.loads` 读取 Value 中的内容，提取出记录的

时间和对应的步数数据或心率数据。原始数据集中的时间为 Unix 时间戳保存，因此数据处理上，先通过 UTC 解析时间戳，然后再转换为北京时间。这部分处理的关键代码如下代码 1 所示。

代码 1　　展开 Value 并转换时间

```
# selected 是已经按记录类型和手环来源筛选的数据
data = pd.json_normalize(selected["Value"].map(json.loads))
data = data[["time", field]]  # field 分别为 steps 或 bpm
data["time"] = pd.to_datetime(data["time"], unit="s", utc=True)
data["time"] = data["time"].dt.tz_convert("Asia/Shanghai")
data["time"] = data["time"].dt.tz_localize(None)
```

检查后发现，同一分钟的步数和心率数据没有重复记录的情况。然后，按时间对心率和步数的两两配对，得到了约一千九百多组有效配对的数据。每组数据中步数和心率的数值都大于零，没有数据错误的部分，所以没有数据被剔除。除了配对成功的部分，剩下的 128 条步数记录和两万多条心率记录因找不到同一分钟的心率或步数数据而配对失败，因此这些记录没有用于聚类。对于缺失的，未记录步数的时刻则继续置空，本文不补全处理。

因此，整理后共得到 1928 组数据，时间跨度自 9 月 20 日到 11 月 2 日之间的记录，即期间共有 26 天留下了可用记录，但每天的记录条数有所不同。下表 2 列出了步数和心率的描述性统计结果，其中，每分钟记录的步数平均为 62.85 步，心率平均为 102.49bpm。由于这批数据只保留了同时记录到步数和心率的时刻，因此后续聚类主要比较不同步数水平下的心率变化。

表 2　配对样本的描述性统计

指标	平均数	标准差	最小值	中位数	最大值
步数	62.85	39.50	1	61	124
心率	102.49	11.56	58	102	148

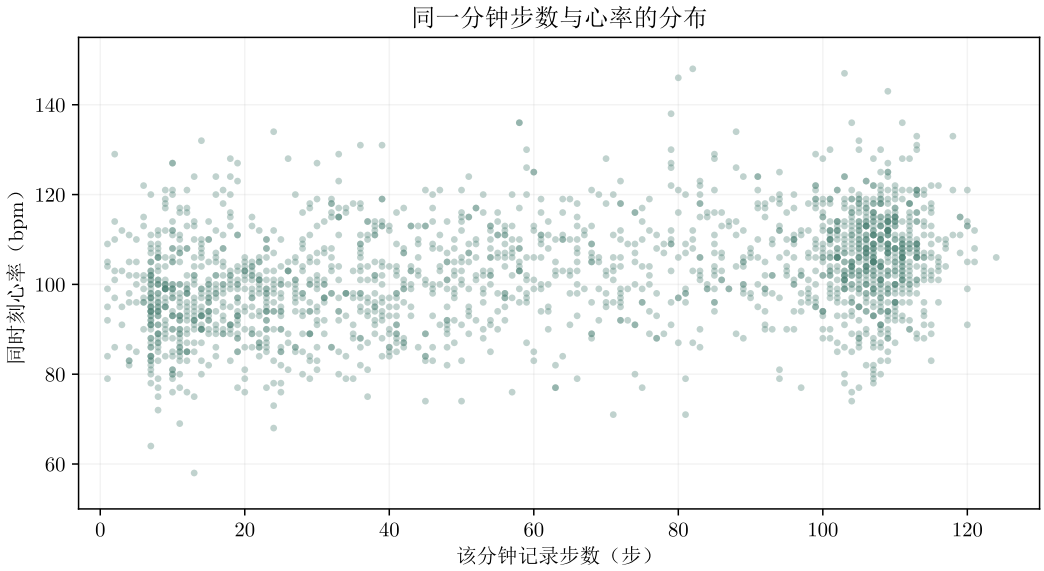


图 1　同一分钟步数与心率的分布

上图 1 展示了同一分钟步数与心率的分布。从图中可以看到，步数较少的区域和步数在每分钟 100 到 110 的区域中都有较多的记录，并且后者的心率整体上是更高的。然而，在固定横轴不变的情况下，即固定步数不变的情况下，同一步数下也能看到具有非常的多的点分布在不同的 y 轴高度上，意味着相同步数下心率仍然有明显的差异，较少的步数也具有较高心率。不过，这个散点图目前虽然可以展现两种指标的组合，但是由于缺少时间轴，所以不好直接看出它们与时间的作用关系，没办法看出一次活动持续了多久，但目前这种方式可以提供一个初步的可视化见解，现在就可以开始进行聚类了，把相近的组合归类到一起，然后再比较它们之间的特点。

三、聚类方法与参数选择

本次采用 K-means 算法，并用 K-means++选择初始中心。算法先把每条记录分配给距离最近的中心，再用组内样本的平均位置更新中心，反复计算，直到分组基本稳定。K-means++改进的是初始中心的选择，让起点尽量分散。分组完成后，再比较各组的步数和心率，这样就可以搞明白不同走动情况下心率的差异。

步数和心率的单位不同，离散程度也不同。配对样本中，步数的标准差为 39.50，心率为 11.56，直接用原始数值计算距离，会让步数的变化占据较大影响。因此先用 `StandardScaler` 对每列减去均值并除以标准差，然后再进行聚类。这样处理过后，两列都按各自的波动幅度参与距离计算，从而减少因单位影响造成的聚类问题。

簇数 k 分别从集合 $\{2,3,4,5\}$ 中取，然后将随机种子固定为 42，n_{init} 设为 10，即每个 k 尝试 10 次初始化，并保留簇内平方和最小的结果。图 2 比较了簇内平方和与轮廓系数，左图反映了组内样本离中心有多远，右图则结合组内紧密程度和组间距离，其中数值越高通常表示分组越清楚。最后，比较这四种分组结果，选择轮廓系数最高的方案，以确定应当将数据分成几组。

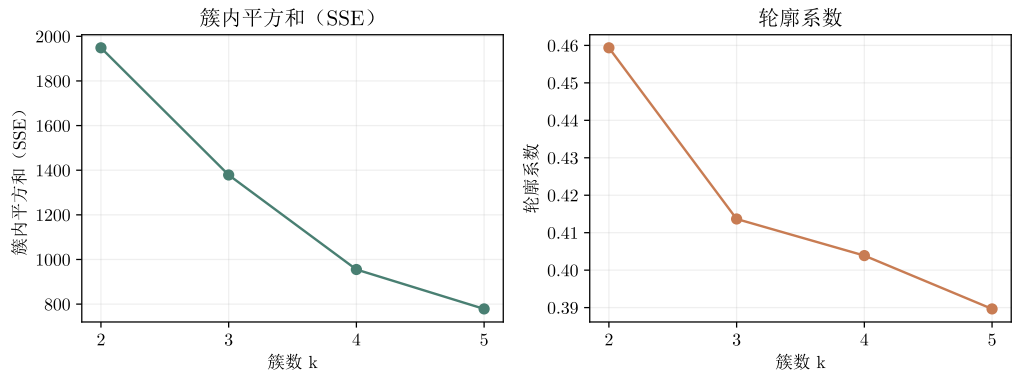


图 2　不同簇数的聚类指标比较

随着 k 增加，簇内平方和不断下降，这是把样本分得更细后常见的情况。轮廓系数从 k 为 2 时的 0.46，降到 k 为 3、4 和 5 时的 0.41、0.40 和 0.39。因此，本次选取 k 为 2，因为它的轮廓系数最高。这个结果表示选择 k 为 2 也就是分成两组时是更合适的。关于标准化和比较候选簇数的关键代码，见如下代码 2 所示。

代码 2　　标准化并比较候选簇数

```
# 两个指标先按各自的均值和标准差进行标准化
x = StandardScaler().fit_transform(data[["steps", "bpm"]])
scores = []
for k in range(2, 6):
    model = KMeans(n_clusters=k, init="k-means++", n_init=10,
random_state=42).fit(x)
    scores.append([k, model.inertia_, silhouette_score(x,
model.labels_)])
```

四、聚类结果与可视化

在选定簇数 $k=2$ 后意味着我们的聚类需要将结果分为两组。将该参数输入模型，重新运行后可以得到最终结果。将平均步数较少的一组记为簇 1，较多的一组记为簇 2，下图 3 显示了聚类模型的聚类结果，下表 3 则是各簇的样本数量和簇中心点的数据。

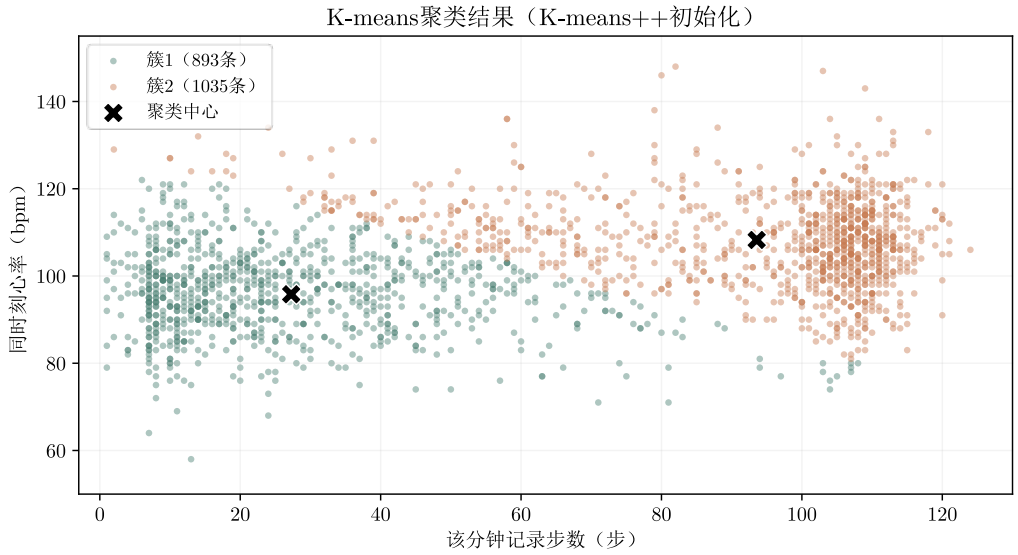


图 3　步数与心率的二维聚类结果

表 3　两个簇的样本数量与中心

簇	记录数	记录占比	平均步数	平均心率
1	893	46.32%	27.26	95.81
2	1035	53.68%	93.56	108.26

簇 1 有 893 条记录，平均步数约为 27 步，平均心率约为 95bpm，因此

簇 1 的划分方式可以理解为步数较少且心率较低。簇 2 有 1035 条记录，平均步数约为 93 步，平均心率约为 108bpm，意味着簇 2 的划分方式是步数较多且心率也比较高的数据。两组中心的心率相差约 12bpm，与图 1 中较多步数对应较高心率的整体变化一致。

图中两组之间仍有不少过渡点，没有明显的空白把它们完全隔开。有些步数很少但心率较高的点被分进簇 2，也有一些步数较多但心率较低的点留在簇 1，说明模型同时考虑了两个指标，而不仅仅局限于只按某个步数界限切分。

图中每分钟约 100 步到 110 步的位置出现了很多记录聚集在一起的情况，心率大多在 90bpm 到 120bpm 之间，构成了一片相对比较密集的区域。步数较少时，心率的差别则更加显著，例如有些情况下只走了十几步，但对应的心率在 80bpm 到 90bpm 附近，甚至有的还超过了 110bpm。将两个指标放在一起观察，除了能看出走动多少，也能注意到这些步数相近但心率却相差较大的记录，后续进一步进行数据分析时，可以根据当天的实际情况进行推断，从而了解这种差别出现的具体原因是什么。

五、分析与讨论

从已有样本来看，我在有计步记录的时刻中既有零散走动，也有每分钟步数较多的片段。后者的平均心率更高，说明增加走动时，心率通常也会有所上升。但本次样本只覆盖部分活动时刻，结合我的实际情况，说明我目前的运动量不是很大，因为样本配对后，平均下来每天同时有走动和心率的分钟数大概是两小时左右。并且，这段时间我没怎么跑步，基本都是上课导致的走动，包括前往第十教学楼等，所以样本本身的覆盖范围可能不足，缺少关于跑动情况下的数据。

针对平时主要靠上课和通勤走动的情况，我计划在课余安排几次专门的锻炼，先从快走开始，选择比较熟悉的路线并记录每次的开始时间，结束时间，以及运动后的感受。同时继续佩戴手环，观察步数增加后心率怎样变化，以及停下来休息后心率的下降情况。这样积累几次记录后，就可以比较相近路线和速度下的心率变化，也能给这些数据打上对应的标签，标记这次活动到底是什么，这样我们不仅仅可以进行聚类的无监督训练，还可以进行基于分类的有监督训练，并评估不同模型，即有监督情况下和无监督情况下模型的运行效果。

六、实验结论

本次实验从小米运动健康导出的明细中，区分了手机与手环数据，展开 Value 并按时间配对，最终得到 1900 多条步数与心率样本。经过标准化和候选簇数比较，采用 K-means++初始化的 K-means 将样本分为两组，分别表现为步数较少但心率相对较低，以及步数较多但心率相对较高这两组的聚类。

聚类帮助我们整理了活动记录中的常见组合，也保留了两组之间的差异和重叠。在未来，可以考虑使用不同的聚类实现并通过轮廓系数评估不同聚类实现在同一数据集上的分类效果，看看它们的可解释力度和分类情况如何。在评估和比较出最好的聚类实现后，就可以为后续的进一步数据分析和体育健康安排提供统计上的一句，更有依据地调整运动安排。

七、附录

analysis.py　程序入口点

```python
from step_one_prepare import run_step_one
from step_two_clustering import run_step_two
from plot import plot_original, plot_selection, plot_clusters

def main() -> None:
    """读取数据、完成聚类并输出三张 SVG。"""
    data = run_step_one()
    plot_original(data)
    result, quality, centers = run_step_two(data)
    plot_selection(quality)
    plot_clusters(result, centers)

if __name__ == "__main__":
    main()
```

common.py　保存公共设置

```python
from pathlib import Path

class Config:
    ROOT: Path = Path(__file__).resolve().parent.parent
    DATA: Path = ROOT / "data"
    FIGURES: Path = ROOT / "figures"
    FEATURES: list[str] = ["steps", "bpm"]
    SEED: int = 42
```

plot.py　绘制实验图表

```python
from pathlib import Path
from typing import Any
import matplotlib.pyplot as plt
import pandas as pd
from common import Config

class PlotConfig:
    STYLE: dict[str, Any] = {
        "font.family": ["Latin Modern Math", "SimSun"],
        "mathtext.fontset": "cm",
        "axes.formatter.use_mathtext": True,
        "axes.unicode_minus": False,
        "svg.fonttype": "path",
        "font.size": 11,
    }
    COLORS: list[str] = ["#497f72", "#c87c53", "#697da0", "#98759a",
"#b4a15b"]
```

```python
plt.rcParams.update(PlotConfig.STYLE)

def save_figure(path: Path) -> None:
    """保存当前图为 SVG。"""
    path.parent.mkdir(exist_ok=True)
    plt.tight_layout()
    plt.savefig(path, format="svg", bbox_inches="tight")
    plt.close()

def set_health_axis(ax: plt.Axes) -> None:
    """统一二维健康散点图的坐标。"""
    ax.set_xlabel("该分钟记录步数（步）")
    ax.set_ylabel("同时刻心率（bpm）")
    ax.set_xlim(-3, 130)
    ax.set_ylim(50, 155)
    ax.grid(alpha=0.15)

def plot_original(data: pd.DataFrame) -> None:
    """展示聚类前的两个指标。"""
    fig, ax = plt.subplots(figsize=(8, 4.5))
    ax.scatter(
        data["steps"],
        data["bpm"],
        s=13,
        alpha=0.35,
        color=PlotConfig.COLORS[0],
        edgecolors="none",
    )
    set_health_axis(ax)
    ax.set_title("同一分钟步数与心率的分布")
    save_figure(Config.FIGURES / "01 二维健康数据.svg")

def plot_selection(quality: pd.DataFrame) -> None:
    """用簇内平方和及轮廓系数比较簇数。"""
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))
    for ax, field, title, color in zip(
        axes,
        ["sse", "silhouette"],
        ["簇内平方和（SSE）", "轮廓系数"],
        PlotConfig.COLORS,
    ):
        ax.plot(quality["k"], quality[field], "o-", color=color)
        ax.set_xticks(quality["k"])
        ax.set_xlabel("簇数 k")
        ax.set_ylabel(title)
        ax.set_title(title)
        ax.grid(alpha=0.2)
    save_figure(Config.FIGURES / "02 聚类簇数选择.svg")

def plot_clusters(result: pd.DataFrame, centers: pd.DataFrame) ->
None:
    """在原始单位下显示聚类结果和中心。"""
    fig, ax = plt.subplots(figsize=(8, 4.5))
    for group, part in result.groupby("cluster"):
```

```
        ax.scatter(
            part["steps"],
            part["bpm"],
            s=13,
            alpha=0.45,
            color=PlotConfig.COLORS[group - 1],
            edgecolors="none",
            label=f"簇{group}（{len(part)}条）",
        )
    ax.scatter(
        centers["steps"],
        centers["bpm"],
        marker="X",
        s=130,
        color="black",
        edgecolors="white",
        linewidths=0.8,
        label="聚类中心",
    )
    set_health_axis(ax)
    ax.set_title("K-means 聚类结果（K-means++初始化）")
    ax.legend(loc="upper left", fontsize=10)
    save_figure(Config.FIGURES / "03 二维健康数据聚类.svg")
```

step_one_prepare.py　　提取并配对数据

```
import json
import pandas as pd
from common import Config

def extract_feature(raw: pd.DataFrame, key: str, field: str, sid:
str) -> pd.DataFrame:
    """读取同一手环的某项记录，展开其中的时间和数值。"""
    selected = raw.query("Key == @key and Sid == @sid")
    data =
pd.json_normalize(selected["Value"].map(json.loads))[["time",
field]]
    data["time"] = pd.to_datetime(data["time"], unit="s", utc=True)
    data["time"] =
data["time"].dt.tz_convert("Asia/Shanghai").dt.tz_localize(None)
    return data

def run_step_one() -> pd.DataFrame:
    """筛选手环记录，按相同时间配对步数和心率。"""
    raw = pd.read_csv(Config.DATA / "fitness_data.csv",
dtype={"Sid": str})
    sources = pd.read_csv(Config.DATA / "data_source.csv",
dtype={"Sid": str})
    sid = sources.loc[sources["Name"].str.strip() == "小米手环 8",
"Sid"].iloc[0]
    steps = extract_feature(raw, "steps", "steps", sid)
    heart = extract_feature(raw, "heart_rate", "bpm", sid)
    print(f"原始文件：{len(raw)}行、{len(raw.columns)}列")
    print(f"手环步数：{len(steps)}条；手环心率：{len(heart)}条")
    print(
        f"重复时间：步数{steps['time'].duplicated().sum()}条，"
```

```
        f"心率{heart['time'].duplicated().sum()}条"
    )

    # 只保留两项指标都有记录的分钟，缺失步数不补为零。
    steps = steps.drop_duplicates("time")
    heart = heart.drop_duplicates("time")
    data = steps.merge(heart, on="time", how="inner")
    clean = data.dropna().query("steps >= 0 and bpm >
0").sort_values("time")
    print(
        f"配对成功：{len(data)}条；缺失或非正心率、负步数剔除：
{len(data) - len(clean)}条"
    )
    print(f"时间范围：{clean['time'].min()} 至
{clean['time'].max()}")
    return clean.reset_index(drop=True)
```

step_two_clustering.py　比较簇数并聚类

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from common import Config

def run_step_two(data: pd.DataFrame) -> tuple[pd.DataFrame,
pd.DataFrame, pd.DataFrame]:
    """标准化两个指标，比较不同簇数后完成聚类。"""
    scaler = StandardScaler()
    x = scaler.fit_transform(data[Config.FEATURES])
    scores = []
    for k in range(2, 6):
        model = KMeans(
            n_clusters=k, init="k-means++", n_init=10,
random_state=Config.SEED
        ).fit(x)
        scores.append([k, model.inertia_, silhouette_score(x,
model.labels_)])
    quality = pd.DataFrame(scores, columns=["k", "sse",
"silhouette"])
    best_k = int(quality.loc[quality["silhouette"].idxmax(), "k"])
    model = KMeans(
        n_clusters=best_k, init="k-means++", n_init=10,
random_state=Config.SEED
    ).fit(x)

    # 恢复原始单位，并按中心步数从少到多给簇编号。
    centers = pd.DataFrame(
        scaler.inverse_transform(model.cluster_centers_),
columns=Config.FEATURES
    )
    order = centers["steps"].sort_values().index
    mapping = {old: new + 1 for new, old in enumerate(order)}
    result = data.copy()
    result["cluster"] = pd.Series(model.labels_).map(mapping)
    centers = centers.loc[order].reset_index(drop=True)
```

```
    centers.index = range(1, best_k + 1)
    print(f"选定簇数：{best_k}；轮廓系数：
{quality['silhouette'].max():.4f}")
    summary = result.groupby("cluster").agg(
        记录数=("bpm", "size"), 平均步数=("steps", "mean"), 平均心率
=("bpm", "mean")
    )
    summary["记录占比（%）"] = summary["记录数"] / len(result) * 100
    summary.index.name = "簇"
    print("聚类结果：\n" + summary.round(2).to_string())
    return result, quality, centers
```